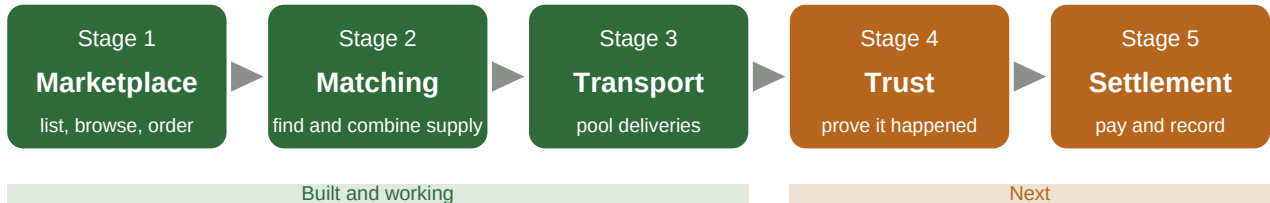


Farmer Marketplace: Stages 1 to 5

What was built in stages 1 to 3, what stages 4 and 5 should contain, and what each stage changes for the project as a whole. Based on *FARMER_MARKETPLACE_BUILD_GUIDE.md* and the current code (2026-10-03).



In short

The project connects farmers who have produce with buyers who need it, and the transporters who move it. Stages 1 to 3 are built: a farmer can list produce, a buyer can ask for it, the system finds and combines nearby supply, and compatible deliveries share a truck. What the system cannot do yet is **prove** that a delivery really happened, or **move money** for it. Those are stages 4 and 5.

- **Stage 4 (trust)** records who confirmed what and when, and builds a reputation from those records. It answers: *can I rely on this farmer, transporter or buyer?*
- **Stage 5 (settlement)** connects payment, and optionally a shared public record on Solana. It answers: *who gets paid, when, and how can everyone verify it?*

The guide's core rule applies to both: the marketplace must keep working without Solana, AI or x402. Stage 4 is plain database work. Stage 5 is an add-on that sits behind an interface, so it can be switched off or replaced.

1. Stages 1 to 3: what exists today

Stage	Question it answers	What a user can do now	Main pieces in the code
1. Marketplace foundation	Can a farmer sell and a buyer buy?	Farmer registers, creates a profile and farm, lists, edits and deactivates produce. Buyer browses, filters by product and price, orders. Admin moderates users and listings and sees all orders.	auth, users, farms, catalog, orders; order state machine in <code>orders/service.py</code> ; PostgreSQL with PostGIS locations
2. Matching and aggregation	Who can fill this order, and from how many farms?	Buyer opens a request (product, quantity, max price, location, radius). The system searches nearby farms, ranks them with an explainable score, and splits the quantity across several farmers. Buyer reviews the match plan and creates one grouped order.	<code>matching/engine.py</code> (radius search, score, greedy fill); <code>match_plans</code> and <code>allocations</code> tables; match-plan screen
3. Pooled transport	Can several deliveries share one truck?	Transporter registers vehicles, finds pools of compatible orders, sees separate versus pooled cost, confirms a pool with a vehicle quote, then confirms pickup and delivery. Farmer marks produce ready; buyer completes.	<code>logistics/</code> (routing interface, pooling heuristic, quotes); <code>vehicles</code> , <code>shipments</code> , <code>shipment_stops</code> , <code>transport_pools</code> , <code>transport_quotes</code> ; order states <code>READY_FOR_PICKUP</code> , <code>IN_TRANSIT</code> , <code>DELIVERED</code>

How the stages build on each other

Stage 1 gives the data (users, farms, lots, orders). Stage 2 turns a buyer's need into an allocation across farmers, which produces the grouped orders. Stage 3 takes those accepted orders and plans how they travel. Each stage reuses the previous one's tables instead of replacing them.

What stages 1 to 3 did for the project

- A working end-to-end marketplace that demonstrates the first seven steps of the guide's demo story (list, request, match, aggregate, pool transport, compare cost, confirm).
- Real data about supply, demand, distances, prices and deliveries, which later stages (and eventually intelligence) depend on.
- A cost argument: in the demo data, pooling cut delivery cost from 22.37 to 13.45 (39.9%).

Known gaps carried into the next stages

The build audit (phase1-3-build-audit.pdf) scored the stages at 70%, 94% and 92%. Four findings matter for stages 4 and 5: the demo shortcut that lets an order complete without delivery (it would defeat delivery evidence), no order cancellation, no external_transactions table, and no domain events module. Section 5 lists them as prerequisites.

2. Stage 4: Trust, evidence and reputation

Objective (from the guide): create useful trust before introducing blockchain. Today the system knows an order was *marked* delivered, but not who confirmed it, when, or with what proof. Stage 4 records that.

What stage 4 should contain

Piece	What it is	Fits into the current code as
Attestations	A signed-off statement: who (actor) confirmed what (type) about which thing (order or shipment), when, with optional evidence URL and hash.	New attestations table. Types such as HANDOFF_CONFIRMED, PICKUP_CONFIRMED, DELIVERY_CONFIRMED, RECEIPT_CONFIRMED.
Confirmation chain	Farmer confirms handoff, transporter confirms pickup, transporter confirms delivery, buyer confirms receipt, then the order completes.	Each existing transition (/ready, /pickup, /deliver, /complete) also writes an attestation by the right actor.
Evidence hashes	A fingerprint (hash) of a photo or document, stored with the attestation. The file stays off-chain; the hash proves it was not changed later.	evidence_url and evidence_hash fields, hash computed server-side.
Reputation events	Facts, not stars: ORDER_COMPLETED, DELIVERY_ON_TIME, DELIVERY_LATE, QUALITY_ACCEPTED, QUALITY_REJECTED, DISPUTE_OPENED, DISPUTE_RESOLVED, ORDER_CANCELLED.	reputation_events table, written when orders reach a final state.
Reputation calculator	The score is a projection of the events, so the formula can change later without losing history.	A function over events per farmer, transporter and buyer; exposed on profiles.
Disputes and transaction history	Either side can open a dispute on an in-transit or delivered order; resolution is recorded; each party can see a verified history.	Uses the existing DISPUTED and REFUNDED states; new dispute endpoints and a history view.
Duplicate protection	The same confirmation cannot be recorded twice.	Unique constraint on (subject, type, actor).

Tests the guide asks for

Reputation event generation, duplicate attestation prevention, dispute cases and evidence hashing.

Prerequisites from today's code

- Remove the ACCEPTED to COMPLETED demo shortcut so an order cannot finish without the delivery attestations.
- Add an in-process domain events module (guide rule 4), so ORDER_DELIVERED can notify reputation without orders importing it.
- Add order cancellation, since ORDER_CANCELLED is a reputation event.

What it changes for the project

- **From claims to records.** Today a status is just a column. After stage 4 it is backed by who confirmed it and when.
- **Better matching input.** The stage 2 score has a fulfilment-risk term that currently looks only at quantity. With reputation, it can reflect a farmer's real delivery history.
- **Disputes become possible to resolve** because there is evidence to look at.
- **Blockchain-ready data.** Attestation hashes are exactly what stage 5 can later anchor on-chain. The guide says this model is deliberately blockchain-ready.

3. Stage 5: Solana settlement adapter

Objective (from the guide): add blockchain only where several parties benefit from a shared, verifiable record. The marketplace database is **not** moved onto Solana. PostgreSQL stays the source of truth; Solana is a settlement and verification layer beside it.

What stage 5 should contain

Piece	What it is	Fits into the current code as
PaymentProvider interface	create_payment, get_status, capture, refund. The order service talks only to this interface, never to Solana code.	New payments module; MockPaymentProvider first, so the whole flow works with no blockchain.
external_transactions table	One generic table for provider, external id, purpose (order, refund), amount, currency, status. Provider-specific ids live here only, not as solana_tx columns across other tables.	New migration; also holds fiat or mock payments.
Payment states and idempotency	Every payment operation carries an idempotency key so a retry can never charge twice; webhook and chain events are verified before changing an order.	payments table plus checks in the payment service.
AttestationProvider	Sends the stage 4 evidence hashes and delivery proofs to the chain when wanted; a no-op version when not.	Interface in the payments or attestations module.
Solana adapter	SolanaPaymentProvider and the attestation client, isolated in one adapter.	Adapter module; the FastAPI app only calls the interface.
Settlement program	A small Anchor (Rust) program for the order or payment commitment, settlement status, delivery proof hashes, dispute outcome and optional reputation checkpoint.	Separate blockchain/solana directory with its own toolchain and tests; not imported into the Python backend.
Chain and database reconciliation	A job that compares chain events with database state and repairs differences; failed transactions are retried safely.	Reconciliation service plus tests with the adapter mocked.

What goes on-chain and what stays off

May go on-chain	Stays off-chain
Order or payment commitment; settlement status; hashes of important delivery attestations; dispute result; optional reputation checkpoint.	Personal data, exact private locations, photos, chat messages, search, routing, raw reviews, large documents, AI output.

Tests the guide asks for

Blockchain adapter mocked in normal backend tests, separate Solana program tests, failed-transaction and retry handling, and chain/database reconciliation.

What it changes for the project

- **Money enters the system.** Until now an order completes with no payment. Stage 5 makes completion trigger payment release, and refunds follow disputes.
- **A shared record no single party controls.** Farmer, buyer and transporter can verify settlement and proofs without trusting the marketplace operator.
- **Optional by design.** The mock and fiat providers keep working, so a deployment can use stage 5 fully, partly, or not at all.
- **New operating cost.** A second toolchain (Rust and Anchor), key management and transaction fees. The guide limits stage 5 to cases where the shared record is worth that.

4. Side by side

	Stage 3 (built)	Stage 4 (trust)	Stage 5 (settlement)
Question	How do goods travel cheaply?	Can I rely on this party, and what proves it?	Who is paid, when, and who can verify it?
New data	Vehicles, pools, shipments, stops, quotes	Attestations, evidence hashes, reputation events, disputes	Payments, external_transactions, on-chain commitments and proof hashes
Main users	Transporters, buyers	All parties, admins resolving disputes	All parties; finance and operations
Technology	PostgreSQL, PostGIS, Python	PostgreSQL, Python (no new infrastructure)	Adds Rust, Anchor, a Solana client, key management
Risk if skipped	Deliveries cost more	No evidence, so disputes and reputation rest on claims	No payment flow; stays a coordination tool
Depends on	Stages 1 and 2	Stage 3 delivery flow; domain events	Stage 4 attestations; PaymentProvider; external_transactions

Why this order

Stage 4 comes first because reputation and evidence are useful on their own and need no new infrastructure. It also produces the data stage 5 wants to anchor: delivery attestations and their hashes. Building stage 5 first would put money on top of records nobody can verify. Doing 4 then 5 also keeps the guide's rule intact: each stage is an addition around a stable core, not a rewrite.

5. Suggested path from here

Step	Work	Why now
0. Clean-up	Disable the demo seed route outside local use; add order cancellation and a way to dissolve a proposed pool; add HTTP-level tests; remove the ACCEPTED to COMPLETED shortcut.	Fixes the audit's high and medium findings and removes things stages 4 and 5 would trip over.
0b. Foundations	Add the in-process domain events module and the external_transactions table.	Stage 4 needs events; stage 5 needs the table. Cheap to add early.
4a. Attestations	attestations table, writes on each delivery transition, duplicate protection, evidence hashing.	Guide milestone D: attestations, evidence hashes.
4b. Reputation	reputation_events, calculator, profile display; optionally feed the fulfilment-risk term in matching.	Guide milestone D: reputation events.
4c. Disputes and history	Open and resolve disputes, transaction history view.	Guide milestone D: disputes, transaction history.
5a. Payment interface	PaymentProvider with a mock implementation, payments and idempotency, wired into order completion.	Guide milestone E: stable PaymentProvider before any Solana code.
5b. Solana adapter and program	Devnet flow, settlement program, delivery proof hash, reconciliation.	Guide milestone E: Solana adapter, testnet flow, settlement program, reconciliation.

After stage 5 the guide continues with intelligence (stage 6) and machine payments with x402 (stage 7). Both are explicitly optional and should wait until real marketplace data exists.